PATIPP.

CREATOR VIDEO / SPEAKING SCRIPT / 3-4 MINUTES

I built a guide for the gaps in my own skills.

Speak naturally. These are your words to adapt, not a speech to memorise.

> Have you ever built something that worked, but wondered whether you could build it again without help? That feeling is why I made PATIPP.
>
> In a recent interview, I created a Spring Boot project and connected its layers. But I used AI for some of the service and controller logic. It showed me a gap: I could follow the code, but I needed more practice writing, explaining and debugging it myself.
>
> I wanted a place I could open and immediately know what to practise next. Something that would guide me from Java fundamentals to building Spring applications, without spending the whole session deciding what to study.
>
> That became PATIPP: a preparation platform where I can work on what I find difficult, keep a record of my attempts, and come back to practise again.

SHOT: Start on camera. Cut to the landing page after introducing PATIPP. Pause briefly after describing the interview.

MAIN SCRIPT / DEMONSTRATION AND CLOSE

Show the practice loop. Keep the story honest.

> Here is the experience I am building for myself. I open the Java and Spring Boot challenge and see the next exercise, a relevant resource, and the behaviour my solution should satisfy.
>
> For example: read some sample JSON and build endpoints to add and delete books. I attempt it in my IDE first. If I get stuck, I can reveal hints progressively. Then I run the checks locally and record whether I completed it independently, with hints, or with a solution.
>
> The part I want to make distinctive is returning to something I needed help with and rebuilding it myself. PATIPP suggests an assisted task to revisit, keeps my last reflection nearby, and records when a later attempt is completed independently. For me, that is a more useful sign of progress than simply ticking a lesson off.
>
> I have also put together a 30-day Java and Spring Boot challenge. I have not started it yet. Electricity issues have interrupted my plans, so I am making room for real life: a ten-to-thirty-minute core exercise when I am busy, with a longer extension when I have more time.
>
> I am not claiming I will master Java in thirty days. I want to become more consistent and more capable of building without depending on generated answers. PATIPP is the guide I am building to help me do that. If you have struggled with the same gap, what would help you come back and practise?

SHOT: Challenge brief → resource and checks → IDE → hint → assistance/reflection → Return & rebuild. Use your actual progress.

SHORT VERSION / 45-60 SECONDS

A shorter take.

> I built PATIPP because an interview showed me a gap in my skills. I could set up a Spring Boot project, but I still needed AI for some of the logic. I wanted to get better at writing and explaining that code myself.
>
> PATIPP gives me a next exercise, relevant resources, progressive hints, and checks to run in my IDE. I record how much help I needed, then come back and try rebuilding the task independently.
>
> I have prepared a thirty-day Java and Spring Boot challenge, but I have not started it yet. Electricity issues have disrupted my schedule. So the plan allows small sessions as well as longer ones.
>
> The goal is simple: less dependence on generated answers, and more confidence in what I can build myself.

Before recording

Keep it real	Show empty progress if you have not started. Do not create a completed challenge for the video.
Demonstrate one task	Use a book API exercise. Show the brief, your IDE, a check, and the assistance/reflection fields.
State the limit	PATIPP records your local checks and self-assessment. It does not execute your Java code.
Google sign-in	Include it in the demo only after a real sign-in works on the deployed site.
A natural ending	Invite feedback about what would make someone return to practise. Avoid promising mastery or a completed journey.